

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

SESSION 3

Optimization techniques, Partitioning the grid into warps and reducing thread divergence

Groothuis Thomas (r0955783)
De Smet Feniks (r0891378)

Academic year 2025–2026

Contents

1 Problem & Goals **2**

2 Methodology **2**

3 Results **3**
3.1 Part 1: . . . 3
3.2 Part 2: . . . 4

4 Conclusion **5**

A Code Implementations **6**
A.1 Part 1: . . . 6
A.1.1 CUDA code . . . 6
A.1.2 Data Collection Script . . . 7
A.2 Part 2: . . . 9
A.2.1 CUDA code . . . 9
A.2.2 Data Collection Scripts . . . 12

1 Problem & Goals

This lab addresses the execution of warps, that are hidden from the programmer and automatically scheduled by the underlying hardware. However, by cleverly designing our algorithms and memory structures, we can align the data in such a way that all threads in a warp follow the same execution path, avoiding thread divergence and the resulting performance penalties.

To find out how this works, we investigate the impact of thread configurations and thread divergence on computation time using an NVIDIA T400 GPU. The goals are divided into two parts:

1. **Grid and Block Scaling:** Implementing an image inversion kernel using a grid-stride loop to evaluate how different combinations of blocks and threads per block influence execution time and hardware occupancy.
2. **Thread Divergence:** Implementing a 1D-Gaussian blur on the red color component. We aim to identify the severe performance penalty caused by thread divergence, which occurs because neighboring threads perform different calculations (a Gaussian blur vs. a simple inversion). We then eliminate this divergence by regrouping the input data by color (RRR...GGG...BBB...), ensuring that all threads within a warp execute the exact same instructions.

2 Methodology

To process a high-resolution image ($960 \times 1280 \times 3$ channels or 3,686,400 elements) both kernels implement a **grid-stride loop**. This ensures that every element is processed exactly once, even if the total number of launched threads is smaller than the input size. Within the kernel each thread calculates its unique global ID and the grid's stride size. A loop then iterates over the data array. Starting at the thread's global ID and incrementing by the stride size, which corresponds to the total number of threads scheduled in the grid. This strategy divides the workload evenly across all active threads. The provided host code flattens the input images into 1D arrays, where every three consecutive integer values represent the interleaved RGB components of a single pixel ($x \in (0, 255)$).

For Part 1 the program calls a kernel that inverts the image. Each thread computes the inverted color component by applying the formula $x = 255 - x$ to its assigned array elements.

For Part 2, the kernel is augmented to apply a 1D-Gaussian blur, which alters the operation of the grid-stride loop. While the green and blue color components are still processed using the standard inversion ($x = 255 - x$), the red component follows a different computational path. Specifically, the ith red pixel is calculated as a Gaussian weighted sum of its 1D-neighboring red pixel values . This is implemented using the following formula:

$$r_i = 0.1r_{i-2} + 0.25r_{i-1} + 0.5r_i + 0.25r_{i+1} + 0.1r_{i+2}$$

To apply this mathematical model to the flattened 1D image array, we must conditionally isolate the red color components to execute the blur arithmetic. Because the 2D image is processed as a 1D array, calculating the blur blindly across edges would erroneously blend pixels from the right edge of one row with pixels from the left edge of the adjacent row. To mitigate these edge-related wrap-around issues and avoid reading outside the array limits, we apply the blur with a strict offset. To satisfy these requirements, the kernel implements a conditional check. It verifies that the element's index i is at least 6, is strictly less than the total input size minus 6 and is perfectly divisible by 3 (i.e., i % 3 == 0). By enforcing this modulo condition, we systematically target the first element of every 3-byte pixel sequence in the interleaved array. This ensures that only the red channels are blurred while safely handling the boundaries.

However, applying this conditional logic directly to the interleaved RGB format is architecturally inefficient. Within the hardware, threads are grouped into warps of 32 threads. Modern GPUs operate on a SIMT (Single Instruction, Multiple Threads) architecture. Due to the interleaved array, adjacent threads in the same warp diverge on branch instructions. The threads assigned to the red channels branch into the higher-complexity blur arithmetic, which takes longer to execute. Meanwhile the adjacent threads handling the green and blue components fall through to the else branch to perform a simple inversion, finishing their job much earlier. Even though they have finished, these threads become idle. They continue to occupy the cores and remain reserved until the entire warp completes the longest execution path. Because these idle cores do not contribute to the processing of data during this waiting period, overall computational performance is degraded.

To resolve this programmatically, we implemented a memory reformatting strategy prior to the kernel launch. The host code pre-sorts the interleaved data array into color planes: all red pixels are grouped together, followed by all green, and finally all blue components (RRR...GGG...BBB...). We then pass this planar structure to a modified kernel. By restructuring the memory layout to align with the hardware's execution model, threads within a single warp will almost exclusively fall into the exact same conditional branch. This ensures homogeneous thread behavior, allowing the kernel to fully utilize the parallel processing power of the GPU without leaving cores idle. This also provides coalesced memory access. Technically, we could have accessed the data differently from within the kernel instead of sorting the array itself, which would result in many separate memory accesses. But that is for the next lab.

To ensure the correctness of our implementations, every execution of these CUDA programs simultaneously benchmarks the kernels and strictly compares the GPU-generated output against a CPU baseline. Execution time is measured using `cudaEvent_t` API calls placed directly around the kernel launches. This placement ensures that the memory transfer overhead between the host and device, which is handled via `cudaMemcpy`, is excluded from the performance metrics. To efficiently collect and aggregate this data, automated Python scripts were utilized. Rather than manually running tests, these scripts automatically execute the compiled programs across 36 different configurations. Sweeping the thread count per block (16 to 1024) and the grid size (1 to 14063 blocks). The Python scripts then parse the standard output to extract the timing results. To filter out operating system pre-emption noise and calculate stable average execution times, the script executes each configuration exactly 20 times.

3 Results

3.1 Part 1:

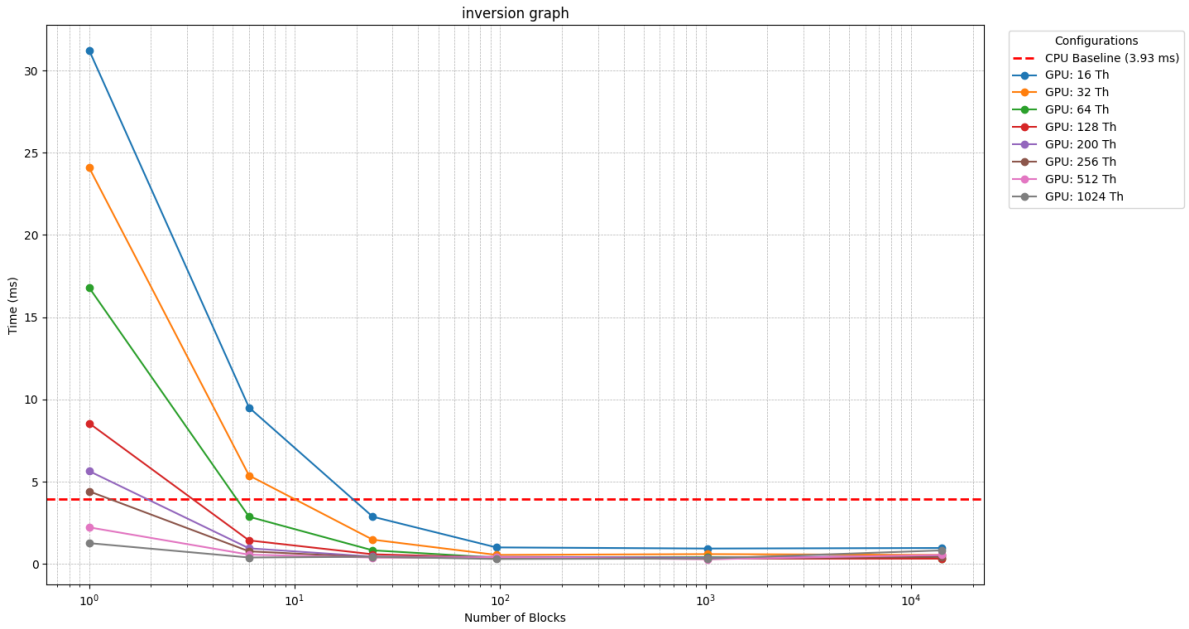


Figure 1: Execution time of the image inversion kernel (Part 1) across different grid and thread configurations compared to the CPU baseline.

blocks/SM	1	2	4	8	16
threads/block	1024	512	256	128	64
warps/block	32	16	8	4	2

Table 1: Valid combinations of blocks and threads per block to maximize hardware utilization on the T400 [1, p. 504].

Before analyzing the results, the preceding table outlines the configurations required to achieve 100% hardware occupancy using only full warps. The NVIDIA T400 GPU utilized in this lab features 6 Streaming Multiprocessors (SMs), each containing 64 CUDA cores. Consequently, a configuration such as 6 blocks with 1024 threads per block theoretically achieves full SM occupancy, though other valid configurations exist.

Figure 1 presents the execution time of the image inversion kernel across various block and thread configurations. As expected, launching only a single block results in the slowest execution times. A single block restricts the workload to just one SM, leaving the remaining five completely idle. If the thread count within this block is lower than 64, even that single SM fails to reach its full parallel capacity. Conversely, assigning more threads to a single block causes the SM to serialize the execution of the active warps. A crucial hardware inefficiency is directly observable when examining partial warp execution. In CUDA architecture, a warp is defined as a strict group of 32 threads. When the configuration launches blocks with only 16 threads, the hardware still schedules a full warp but leaves half of the allocated threads idle. These idle threads occupy cores without contributing to the processing of data. This penalty is clearly visible in the graph: the 16 threads/block configuration (blue curve) is consistently slower than the 32 threads/block configuration (orange curve) across the grid sizes. Because 50% of the computational power within every scheduled warp is wasted, the grid-stride loop requires significantly more time to process the array. This exact same architectural inefficiency occurs with any thread count that is not a clean multiple of 32. For instance, the 200 threads/block configuration (purple curve) dictates that the GPU must allocate 7 full warps (224 threads) per block to accommodate the 6.25 warps worth of data. Consequently, the final warp contains only 8 active threads, leaving 24 cores idle. While the performance penalty is less severe than the 16-thread configuration, it inherently prevents the GPU from reaching optimal utilization compared to perfectly aligned powers of two, such as 128 or 256 threads. Interestingly, the performance curves begin to flatten when the overall grid configuration provides enough total active warps to keep the 64 cores within each SM fully fed. High occupancy is not strictly dependent on having a massive thread count within a single block. Rather it relies on distributing a sufficient total number of threads across all available SMs. For instance, launching 6 blocks with 1024 threads per block shows a massive improvement over a single block because all 6 SMs receive a heavy workload simultaneously. However, launching a much higher number of smaller blocks (e.g., 96 blocks of 64 threads) can similarly achieve full occupancy, as the SM schedulers simply distribute these smaller blocks across the available resources to hide memory latency and prevent cores from idling. Ultimately, the data shows that launching an insufficient number of blocks (e.g., 1 or 6) drastically increases execution time, as the 6 SMs of the T400 are underutilized and the grid-stride loop is forced to do multiple iterations per thread. The sweet spot is reached when launching enough blocks to fully saturate the GPU, with a thread-per-block count of 128 or 256 providing optimal warp scheduling without fractional waste.

3.2 Part 2:

In Part 2 the implementation utilizes an if/else condition that introduces severe thread divergence within the warps. Essentially, the threads processing the blue and green pixel values (simple inversion) finish significantly faster than the threads calculating the red pixel values (complex Gaussian blur). Because a warp manages hardware resources as a collective unit, the cores assigned to the idle blue and green threads remain reserved and unproductive until the slower red threads complete their task. In the interleaved RGB configuration, this results in "mixed bag" warps containing heterogeneous workloads, leading to much slower execution times. Conversely, sorting the data into a planar RRR GGG BBB configuration creates homogeneous warps. If the threads calculating the red pixels and those calculating the others are cleanly separated into groups of 32. The threads within a single warp run for an equal amount of time and thus eliminate thread divergence inside the warp. Furthermore, these slow, mixed warps ultimately delay the execution of the block itself. A block is only marked as finished when its slowest warp is done. While having multiple active blocks on a Streaming Multiprocessor (SM) might seem to mitigate this, it becomes a severe bottleneck due to the maximum block capacity of an SM. A block that is kept alive by a single slow, divergent warp occupies a slot on the SM, actively preventing new blocks from being loaded from the pending queue. The worst-case scenario for these inefficiencies is observable in the unsorted 16 threads/block configuration. This setup suffers a "double penalty": it inherently wastes 50% of its computational power due to partial warps (as established in Part 1), while simultaneously suffering from severe thread divergence within the active half of the warp. While launching a massive number of blocks onto the GPU successfully flattens the execution curve by maximizing hardware occupancy, it does not eliminate the negative effects of thread divergence. Figure 2 perfectly highlights this permanent impact. The baseline CPU execution hovered around 9.68 ms.

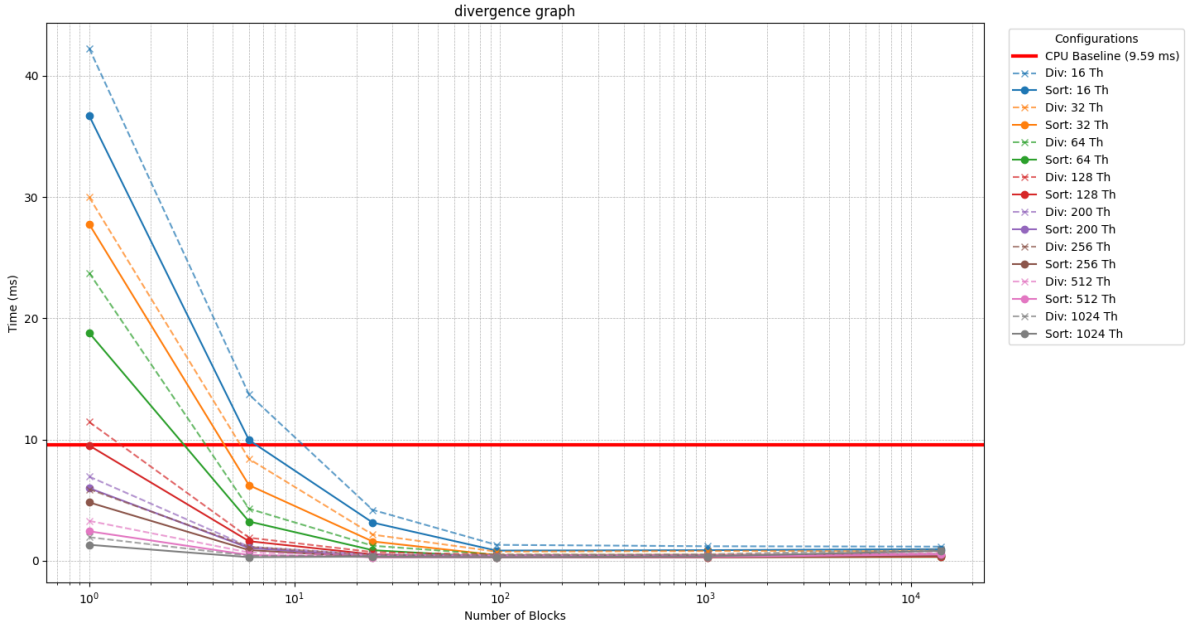


Figure 2: Comparison of execution times for the 1D-Gaussian blur (Part 2) with thread divergence (interleaved data) and without thread divergence (planar data).

The divergent GPU kernel (operating on interleaved RGB) reduced this to approximately 0.88 ms at peak occupancy. However, by simply reorganizing the data into a planar array and running the sorted kernel, the execution time dropped further to ∼0.62 ms. This proves an execution speedup of roughly 30% purely by eliminating warp divergence, a performance gap that persists regardless of how many blocks are launched.

4 Conclusion

This lab successfully demonstrates the critical importance of understanding underlying GPU hardware architecture when writing CUDA code. While leveraging the GPU provides massive speedups compared to a CPU (accelerating a task from 9.6 ms to sub-millisecond ranges), naive implementations leave significant performance on the table. We verified that maintaining a 1D grid-stride loop creates highly flexible kernels that adapt to any given block and thread configuration. However, achieving peak performance requires adhering to three core architectural principles derived from our results. First, conditional logic with unequal execution times induces severe thread divergence inside warps. Because hardware resources are managed collectively, diverging execution paths force faster threads to wait unproductively. Second, this imbalance inherently affects execution at the block level. A CUDA block cannot be retired from the Streaming Multiprocessor (SM) until its slowest warp finishes. Consequently, warps with unequal execution times cause their parent block to unnecessarily occupy a block reserve, actively preventing new blocks from being scheduled. Our results empirically validate that this divergence is a major bottleneck. By optimizing our memory layout (sorting into a planar RGB format) to group threads into homogeneous multiples of 32, we ensured uniform thread behavior. This minor architectural adjustment eliminated the internal wait times and yielded a 30% performance increase. Third, to fully capitalize on the GPU's parallel processing power, SMs must be filled to maximum occupancy. As demonstrated in our theoretical configuration table, this requires launching valid combinations of blocks constructed from clean multiples of 32 threads. Aligning the grid configuration with the hardware limits prevents the heavy penalties of partial warps and keeps the cores fully saturated. Ultimately, these findings underscore that data layout, branch alignment, and precise hardware configurations are just as important as the raw computational logic in GPU programming.

A Code Implementations

A.1 Part 1:

A.1.1 CUDA code

```
/* * Code snippet for importing / exporting image data.
* * To convert an image to a pixel map, run ‘convert <name>.<extension> <name>.ppm
* */
#include <cstdint> // Data types
#include <iostream> // File operations
#include <cstdlib> // atoi
#include <cuda_runtime.h> // CUDA timing

// #define M 512 // Lenna width
// #define N 512 // Lenna height
#define M 960 // VR width
#define N 1280 // VR height
#define C 3 // Colors
#define OFFSET 16 // Header length


uint8_t* get_image_array(void){
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./anna.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    // Initialize empty image array
    uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);
    // Read the image
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET, imageFile);
    // Close the file
    fclose(imageFile);
    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}


void save_image_array(const char* filename, uint8_t* image_array){
    FILE *imageFile;
    imageFile = fopen(filename, "wb");
    if(imageFile == NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    fprintf(imageFile,"P6\n");
    fprintf(imageFile,"%d %d\n", M, N);
    fprintf(imageFile,"255\n");
    fwrite(image_array, 1, M*N*C, imageFile);
    fclose(imageFile);
}

__global__ void invertPixelComponent(uint8_t* d_img, int input_size) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for (int i = idx; i <input_size; i += stride) {
        d_img[i] =255 - d_img[i];
    }
}



int main (int argc, char* argv[]) {
    if (argc != 3) {
        std::cout << "Usage: ./session3 <num_blocks> <threads_per_block>\n";
        return 1;
    }

    int blocks = std::atoi(argv[1]);
```

```
    int threadsPerBlock = std::atoi(argv[2]);

    int input_size = M * N * C;

    uint8_t* image_array = get_image_array();
    uint8_t* new_image_array_cpu = (uint8_t*)malloc(input_size);
    uint8_t* new_image_array_gpu = (uint8_t*)malloc(input_size);
    // ---------------- CPU Timing ----------------
    cudaEvent_t startCPU, stopCPU;
    cudaEventCreate(&startCPU);
    cudaEventCreate(&stopCPU);

    cudaEventRecord(startCPU);

    std::cout << M << " " << N <<" " << C << std::endl;
    for(int i=0; i<input_size; i++){
        new_image_array_cpu[i] = 255-image_array[i];
    }
    cudaEventRecord(stopCPU);
    cudaEventSynchronize(stopCPU);

    float cpuTime = 0.0f;
    cudaEventElapsedTime(&cpuTime, startCPU, stopCPU);
    std::cout << "CPU Time: " << cpuTime << " ms" << std::endl;

    cudaEventDestroy(startCPU);
    cudaEventDestroy(stopCPU);

    // ---------------- GPU Execution & Timing ----------------
    uint8_t *d_img;
    cudaMalloc(&d_img, input_size * sizeof(uint8_t));

    cudaMemcpy(d_img, image_array, input_size * sizeof(uint8_t), cudaMemcpyHostToDevice);

    cudaEvent_t startGPU, stopGPU;
    cudaEventCreate(&startGPU);
    cudaEventCreate(&stopGPU);

    cudaEventRecord(startGPU);
    invertPixelComponent<<<blocks, threadsPerBlock>>>(d_img, input_size);
    cudaDeviceSynchronize();

    cudaEventRecord(stopGPU);
    cudaEventSynchronize(stopGPU);

    float gpuTime = 0.0f;
    cudaEventElapsedTime(&gpuTime, startGPU, stopGPU);
    std::cout << "GPU Time: " << gpuTime << " ms (Blocks: " << blocks << ", Threads/Block: "
    << threadsPerBlock << ")" << std::endl;

    cudaMemcpy(new_image_array_gpu, d_img, input_size * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);

    for (int i=0; i <input_size; i++) {
        if(new_image_array_cpu[i] != new_image_array_gpu[i]) {
            std::cout << "Images are not the same" << std::endl;
            break;
        }
    }

    cudaFree(d_img);

    save_image_array("./output_cpu.ppm", new_image_array_cpu);

    save_image_array("./output_gpu.ppm", new_image_array_gpu);
    return 0;
}
```

A.1.2 Data Collection Script

```
import subprocess
import csv
```

```
import re
import matplotlib.pyplot as plt

thread_counts = [16,32, 64, 128,200, 256, 512, 1024]

block_counts = [1, 6, 24, 96, 1024, 14063]

runs_per_config = 20
csv_file = "results_session3.csv"

results = []

for t in thread_counts:
    for b in block_counts:
        print(f"Running Blocks = {b:>5}, Threads/Block = {t:>4} ...", end=" ")

    cpu_total = 0.0
    gpu_total = 0.0
    valid_runs = 0

    for _ in range(runs_per_config):
        result = subprocess.run(["./session3-1", str(b), str(t)],
            capture_output=True,text=True)

        output = result.stdout

        cpu_match = re.search(r"CPU Time:\s*([0-9.]+)", output)
        gpu_match = re.search(r"GPU Time:\s*([0-9.]+)", output)

        if cpu_match and gpu_match:
            cpu_total += float(cpu_match.group(1))
            gpu_total += float(gpu_match.group(1))
            valid_runs += 1
        else:
            print(f"\nWarning: couldn't parse output: {output}")

    if valid_runs > 0:
        cpu_avg = cpu_total / valid_runs
        gpu_avg = gpu_total / valid_runs
        results.append((b, t, cpu_avg, gpu_avg))
        print(f"CPU: {cpu_avg:.2f} ms | GPU: {gpu_avg:.2f} ms")
    else:
        print("No suitable data for this run.")

with open(csv_file, mode='w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(["Blocks", "ThreadsPerBlock", "CPU_ms", "GPU_ms"])
    for row in results:
        writer.writerow(row)

print(f"\nResults saved to {csv_file}")

plt.figure(figsize=(10, 6))

overall_cpu_avg = sum(r[2] for r in results) / len(results)

plt.axhline(y=overall_cpu_avg, color='red', linestyle='--', linewidth=2,
    label=f"CPU Baseline ({overall_cpu_avg:.2f} ms)")

for t in thread_counts:
    x_blocks = [r[0] for r in results if r[1] == t]
    y_gputime = [r[3] for r in results if r[1] == t]
    if x_blocks:
        plt.plot(x_blocks, y_gputime, marker='o', label=f"GPU: {t} Threads/Block")

plt.xlabel("Number of blocks")
plt.ylabel("Time (ms)")
plt.title(f"CUDA Image Inversion Benchmark ({runs_per_config} runs per config)")
plt.xscale("log")
plt.grid(True, which="both", ls="--", lw=0.5)
plt.legend(title="Legend")
plt.tight_layout()
```

```
plt.show()
```

Listing 1: Automated Python benchmark script

A.2 Part 2:

A.2.1 CUDA code

```
/* * Code snippet for importing / exporting image data.
* * To convert an image to a pixel map, run `convert <name>.<extension> <name>.ppm
* */
#include <cstdint> // Data types
#include <iostream> // File operations
#include <cstdlib> // atoi
#include <cuda_runtime.h> // CUDA timing

// #define M 512 // Lenna width
// #define N 512 // Lenna height
#define M 960 // VR width
#define N 1280 // VR height
#define C 3 // Colors
#define OFFSET 16 // Header length


uint8_t* get_image_array(void){
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./anna.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    // Initialize empty image array
    uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);
    // Read the image
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET, imageFile);
    // Close the file
    fclose(imageFile);
    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}

void save_image_array(const char* filename, uint8_t* image_array){
    FILE *imageFile;
    imageFile = fopen(filename, "wb");
    if(imageFile == NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    fprintf(imageFile,"P6\n");
    fprintf(imageFile,"%d %d\n", M, N);
    fprintf(imageFile,"255\n");
    fwrite(image_array, 1, M*N*C, imageFile);
    fclose(imageFile);
}

__global__ void gaussianBlur(const uint8_t* d_in, uint8_t* d_out, int input_size) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for (int i = idx; i < input_size; i += stride) {
        if (i >= 6 && i < input_size - 6 && i % 3 == 0) {
            d_out[i] = 0.1f * d_in[i-6] + 0.25f * d_in[i-3] + 0.5f * d_in[i] +
                       0.25f * d_in[i+3] + 0.1f * d_in[i+6];
        } else {
            d_out[i] = 255 - d_in[i];
        }
    }
}

__global__ void gaussianBlurSorted(const uint8_t* d_in, uint8_t* d_out, int num_pixels) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
```

```
    int stride = blockDim.x * gridDim.x;
    int total_size = num_pixels * 3;

    for (int i = idx; i < total_size; i += stride) {
        if (i < num_pixels) {
            if (i >= 2 && i < num_pixels - 2) {
                d_out[i] = 0.1f * d_in[i-2] + 0.25f * d_in[i-1] + 0.5f * d_in[i] +
                           0.25f * d_in[i+1] + 0.1f * d_in[i+2];
            } else {
                d_out[i] = 255 - d_in[i];
            }
        } else {
            d_out[i] = 255 - d_in[i];
        }
    }
}

int main (int argc, char* argv[]) {
    if (argc != 3) {
        std::cout << "Usage: ./session3 <num_blocks> <threads_per_block>\n";
        return 1;
    }

    int blocks = std::atoi(argv[1]);
    int threadsPerBlock = std::atoi(argv[2]);

    int num_pixels = M * N;
    int input_size = num_pixels * C;

    uint8_t* image_array = get_image_array();
    uint8_t* new_image_array_cpu = (uint8_t*)malloc(input_size);
    uint8_t* new_image_array_gpu_divergent = (uint8_t*)malloc(input_size);
    uint8_t* new_image_array_gpu_sorted = (uint8_t*)malloc(input_size);
    // ========================================================================
    // 1. CPU RUN (Baseline)
    // ========================================================================
    cudaEvent_t startCPU, stopCPU;
    cudaEventCreate(&startCPU);
    cudaEventCreate(&stopCPU);

    cudaEventRecord(startCPU);
    for(int i = 0; i < input_size; i++){
        if (i >= 6 && i < input_size - 6 && i % 3 == 0) {
            new_image_array_cpu[i] = 0.1f*image_array[i-6] + 0.25f*image_array[i-3] +
                                     0.5f*image_array[i] + 0.25f*image_array[i+3] +
                                     0.1f*image_array[i+6];
        } else {
            new_image_array_cpu[i] = 255 - image_array[i];
        }
    }
    cudaEventRecord(stopCPU);
    cudaEventSynchronize(stopCPU);

    float cpuTime = 0.0f;
    cudaEventElapsedTime(&cpuTime, startCPU, stopCPU);
    std::cout << "CPU Time: " << cpuTime << " ms" << std::endl;

    // ========================================================================
    // 2. GPU RUN 1: Divergent
    // ========================================================================
    uint8_t *d_in, *d_out;
    cudaMalloc(&d_in, input_size * sizeof(uint8_t));
    cudaMalloc(&d_out, input_size * sizeof(uint8_t));

    cudaMemcpy(d_in, image_array, input_size * sizeof(uint8_t), cudaMemcpyHostToDevice);

    cudaEvent_t startGPU1, stopGPU1;
    cudaEventCreate(&startGPU1);
    cudaEventCreate(&stopGPU1);

    cudaEventRecord(startGPU1);
    gaussianBlur<<<blocks, threadsPerBlock>>>(d_in, d_out, input_size);
    cudaDeviceSynchronize();
```

```
136     cudaEventRecord(stopGPU1);
137     cudaEventSynchronize(stopGPU1);
138 
139     float gpuTimeDivergent = 0.0f;
140     cudaEventElapsedTime(&gpuTimeDivergent, startGPU1, stopGPU1);
141     std::cout << "GPU Time Divergent: " << gpuTimeDivergent << " ms" << std::endl;
142 
143     cudaMemcpy(new_image_array_gpu_divergent, d_out, input_size * sizeof(uint8_t),
        cudaMemcpyDeviceToHost);
144 
145     // ============================================================================
146     // 3. GPU RUN 2: Sorted
147     // ============================================================================
148     uint8_t* sorted_image_array = (uint8_t*)malloc(input_size);
149     for(int i = 0; i < num_pixels; i++) {
150         sorted_image_array[i] = image_array[i * 3];
151         sorted_image_array[num_pixels + i] = image_array[i * 3 + 1];
152         sorted_image_array[2 * num_pixels + i] = image_array[i * 3 + 2];
153     }
154 
155     cudaMemcpy(d_in,sorted_image_array, input_size * sizeof(uint8_t), cudaMemcpyHostToDevice)
        ;
156 
157     cudaEvent_t startGPU2, stopGPU2;
158     cudaEventCreate(&startGPU2);
159     cudaEventCreate(&stopGPU2);
160 
161     cudaEventRecord(startGPU2);
162     gaussianBlurSorted<<<blocks, threadsPerBlock>>>(d_in, d_out, num_pixels);
163     cudaDeviceSynchronize();
164     cudaEventRecord(stopGPU2);
165     cudaEventSynchronize(stopGPU2);
166 
167     float gpuTimeSorted = 0.0f;
168     cudaEventElapsedTime(&gpuTimeSorted, startGPU2, stopGPU2);
169     std::cout << "GPU Time Sorted: " << gpuTimeSorted << " ms" << std::endl;
170 
171     uint8_t* temp_gpu_sorted_out = (uint8_t*)malloc(input_size);
172     cudaMemcpy(temp_gpu_sorted_out, d_out, input_size * sizeof(uint8_t),
        cudaMemcpyDeviceToHost);
173 
174     for(int i = 0; i < num_pixels; i++) {
175         new_image_array_gpu_sorted[i * 3] = temp_gpu_sorted_out[i];
176         new_image_array_gpu_sorted[i * 3 + 1] = temp_gpu_sorted_out[num_pixels + i];
177         new_image_array_gpu_sorted[i * 3 + 2] = temp_gpu_sorted_out[2 * num_pixels + i];
178     }
179 
180     bool matchDiv = true;
181     bool matchSort = true;
182     for (int i = 0; i < input_size; i++) {
183         if(abs(new_image_array_cpu[i] - new_image_array_gpu_divergent[i]) > 1) {
184             matchDiv = false;
185         }
186         if(abs(new_image_array_cpu[i] - new_image_array_gpu_sorted[i]) > 1) {
187             matchSort = false;
188         }
189     }
190     std::cout << "Validatie Divergent: " << (matchDiv ? "SUCCES" : "FAILED") << std::endl;
191     std::cout << "Validatie Sorted: " << (matchSort ? "SUCCES" : "FAILED") << std::endl;
192 
193     cudaFree(d_in);
194     cudaFree(d_out);
195     cudaEventDestroy(startCPU);
196     cudaEventDestroy(stopCPU);
197     cudaEventDestroy(startGPU1);
198     cudaEventDestroy(stopGPU1);
199     cudaEventDestroy(startGPU2);
200     cudaEventDestroy(stopGPU2);
201 
202     free(sorted_image_array);
203     free(temp_gpu_sorted_out);
204     free(new_image_array_cpu);
205     free(new_image_array_gpu_divergent);
```

```
    free(new_image_array_gpu_sorted);
    free(image_array - OFFSET);

    return 0;
}
```

A.2.2 Data Collection Scripts

```
import subprocess
import csv
import re
import matplotlib.pyplot as plt

thread_counts = [16, 32, 64, 128, 200, 256, 512, 1024]
block_counts = [1, 6, 24, 96, 1024, 14063]

runs_per_config = 20
csv_file = "results_session3_2.csv"


results = []

for t in thread_counts:
    for b in block_counts:
        print(f"Running Blocks = {b:>5}, Threads/Block = {t:>4} ...", end=" ")

        cpu_total = 0.0
        gpu_div_total = 0.0
        gpu_sort_total = 0.0
        valid_runs = 0

        for _ in range(runs_per_config):
            result = subprocess.run(["./session3-2", str(b), str(t)],capture_output=True,
                                    text=True
            )

            output = result.stdout

            cpu_match = re.search(r"CPU Time:\s*([0-9.]+)", output)
            gpu_div_match = re.search(r"GPU Time Divergent:\s*([0-9.]+)", output)
            gpu_sort_match = re.search(r"GPU Time Sorted:\s*([0-9.]+)", output)

            if cpu_match and gpu_div_match and gpu_sort_match:
                cpu_total += float(cpu_match.group(1))
                gpu_div_total += float(gpu_div_match.group(1))
                gpu_sort_total += float(gpu_sort_match.group(1))
                valid_runs += 1
            else:
                print(f"\nWarning: couldn't parse output: {output}")

        if valid_runs > 0:
            cpu_avg = cpu_total / valid_runs
            gpu_div_avg = gpu_div_total / valid_runs
            gpu_sort_avg = gpu_sort_total / valid_runs
            results.append((b, t, cpu_avg, gpu_div_avg, gpu_sort_avg))
            print(f"CPU: {cpu_avg:.2f} ms | Div: {gpu_div_avg:.2f} ms
                    | Sort: {gpu_sort_avg:.2f} ms")
        else:
            print("No suitable data for this run.")

with open(csv_file, mode='w', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(["Blocks", "ThreadsPerBlock", "CPU_ms", "GPU_Divergent_ms", "
    GPU_Sorted_ms"])
    for row in results:
        writer.writerow(row)

print(f"\nResults saved to {csv_file}")

plt.figure(figsize=(12, 8))

overall_cpu_avg = sum(r[2] for r in results) / len(results)
```

```
64
65 plt.axhline(y=overall_cpu_avg, color='red', linestyle='-', linewidth=3,
66 label=f"CPU Baseline ({overall_cpu_avg:.2f} ms)")
67
68 colors = plt.cm.tab10.colors
69
70 for i, t in enumerate(thread_counts):
71     c = colors[i % len(colors)]
72     x_blocks = [r[0] for r in results if r[1] == t]
73     y_div = [r[3] for r in results if r[1] == t]
74     y_sort = [r[4] for r in results if r[1] == t]
75     if x_blocks:
76     plt.plot(x_blocks, y_div, marker='x', linestyle='--', color=c, alpha=0.7,
77              label=f"Div: {t} Th")
78     plt.plot(x_blocks, y_sort, marker='o', linestyle='-', color=c, label=f"Sort: {t} Th")
79
80 plt.xlabel("Amount of Blocks")
81 plt.ylabel("Time (ms)")
82 plt.title(f"Thread Divergence Impact on T400 ({runs_per_config} runs per config)")
83 plt.xscale("log")
84 plt.grid(True, which="both", ls="--", lw=0.5)
85
86 plt.legend(title="Configurations", bbox_to_anchor=(1.02, 1), loc='upper left')
87 plt.tight_layout()
88 plt.show()
```

Listing 2: Automated Python benchmark script

References

[1] NVIDIA Corporation. *CUDA C++ Programming Guide*. This was used as a basis to calculate the values of the table.

[2] TechPowerUp. *NVIDIA T400 4 GB Specs - GPU Database*. `https://www.techpowerup.com/gpu-specs/t400-4-gb.c3843`